

SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 2

ERROR ANALYSIS TASK

Name of the Student	D.N Shirshika
Reg. No	192425396

COURSE NAME: Deep Learning
FACULTY : Dr.Arul Raj A.M

COURSE CODE: MLA0403

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 30%

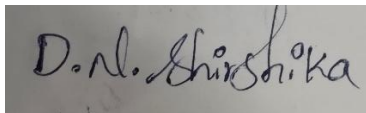
Duration: 30 minutes

Marks: 25

Code of Conduct:

I D.N Shirshika (192425396) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

QUESTIONS:

Total Marks:25

1. Error Analysis in Maximum Likelihood Estimation (MLE)

A student builds a binary classifier using Maximum Likelihood Estimation. The likelihood function is incorrectly written as the sum of probabilities instead of the product. The model produces poor parameter estimates even with large data.

Tasks:

a) Identify the conceptual error in the likelihood formulation.

Answer :

he conceptual error is that the likelihood function is written as the sum of probabilities instead of the product of probabilities of all independent observations.

Incorrect Formulation:

$$L(\theta) = \sum_{i=1}^n P(x_i | \theta)$$

Correct Formulation:

$$L(\theta) = \prod_{i=1}^n P(x_i | \theta)$$

Since observations are assumed to be independent, their probabilities must be multiplied to obtain the likelihood of the entire dataset.

b) Explain how this mistake affects parameter estimation.

Answer :

Using the sum of probabilities instead of the product leads to an incorrect likelihood function and therefore incorrect parameter estimates.

Effects:

- Does not represent the joint probability of the dataset.
- Produces biased or inaccurate parameter estimates.
- The model fails to identify the true maximum likelihood parameters.
- Classification performance decreases.
- Prediction accuracy remains poor even with large amounts of data.

As a result, the classifier may not learn the actual relationship between the features and class labels.

c) Suggest the correct mathematical formulation and reasoning.

Answer :

For independent observations $x_1, x_2, \dots, x_n$, the likelihood function is:

$$L(\theta) = \prod_{i=1}^n P(x_i | \theta)$$

The Maximum Likelihood Estimate (MLE) is obtained by:

$$\hat{\theta} = \arg \max_{\theta} L(\theta)$$

Reasoning:

- The likelihood measures the probability of observing the entire dataset.
- Independent probabilities must be multiplied according to probability theory.
- Maximizing the likelihood gives the parameter values that best explain the observed data.

Therefore, the product formulation correctly captures the statistical behavior of the dataset.

d) How would using log-likelihood fix computational issues?

Answer :

he log-likelihood is obtained by taking the logarithm of the likelihood function:

$$\log L(\theta) = \sum_{i=1}^n \log P(x_i | \theta)$$

Using log-likelihood provides several advantages:

- Converts multiplication into addition, making calculations simpler.
- Prevents numerical underflow caused by multiplying many small probabilities.
- Makes differentiation easier during optimization.
- Improves computational efficiency.
- Helps gradient-based optimization algorithms converge more effectively.

Therefore, log-likelihood is widely used in machine learning because it simplifies computations while preserving the location of the maximum likelihood estimate.

2. Error Diagnosis in Perceptron Learning

While implementing a Perceptron, a student observes that the model fails to converge even after many iterations on a linearly separable dataset.

Tasks:

a) List possible implementation errors in the learning algorithm.

Answer :

Several implementation errors can prevent a Perceptron from converging:

- Incorrect weight update formula.
- Missing bias term in the model.
- Wrong activation function implementation.
- Incorrect label encoding (e.g., using 0 and 1 instead of -1 and +1 when required).
- Errors in prediction calculation.
- Sign mistakes in weight updates.
- Incorrect initialization of weights.
- Failure to update weights after misclassification.

These errors can cause the model to learn an incorrect decision boundary.

b) Analyze how incorrect learning rate or weight updates affect convergence.

Answer :

The Perceptron updates its weights using:

$$w_{new} = w_{old} + \eta(y - \hat{y})x$$

where:

- η = learning rate
- y = actual output
- $\hat{y}$ = predicted output

Effects of an Incorrect Learning Rate:

Very Small Learning Rate

- Weight changes are tiny.
- Learning becomes very slow.
- Requires many iterations to converge.

Very Large Learning Rate

- Weight updates become too large.
- The algorithm may overshoot the optimal solution.
- Causes oscillations and unstable learning.

Incorrect Weight Updates

- Decision boundary moves in the wrong direction.
- Errors accumulate during training.
- The model may never converge even on linearly separable data.

c) Identify dataset-related issues that may cause this problem.

Answer :

Even if the dataset appears linearly separable, the following issues may prevent convergence:

- Presence of noisy or mislabeled samples.
- Outliers affecting the decision boundary.
- Data not properly normalized or scaled.
- Class imbalance.

- Duplicate or inconsistent training samples.
- Missing values or data preprocessing errors.

These factors can make learning difficult and slow down convergence.

d) Propose modifications to ensure convergence.

Answer :

he following modifications can improve convergence:

1. Use the correct weight update rule.
2. Choose an appropriate learning rate.
3. Include a bias term in the model.
4. Normalize or standardize input features.
5. Remove noisy samples and outliers.
6. Verify the correctness of class labels.
7. Shuffle training data during each epoch.
8. Initialize weights properly.
9. Monitor training error and adjust parameters if needed.

By applying these modifications, the Perceptron can successfully learn a decision boundary and converge on linearly separable datasets.

Conclusion

The Perceptron algorithm is guaranteed to converge on a linearly separable dataset if it is implemented correctly. Failure to converge usually results from implementation mistakes, inappropriate learning rates, incorrect weight updates, or dataset-related issues such as noise and outliers. Proper preprocessing, correct parameter settings, and accurate weight update mechanisms are essential for achieving successful convergence and reliable classification performance.

3. Backpropagation Gradient Error Analysis

A Multilayer Perceptron trained using Backpropagation shows increasing loss during training instead of decreasing.

Tasks:

a) Identify possible mathematical or coding errors in gradient computation.

Answer :

Several mathematical or coding mistakes can cause the loss to increase instead of decrease during training:

- Incorrect derivative calculations of activation functions.
- Wrong implementation of the chain rule.
- Sign errors in gradient computation.
- Incorrect loss function calculation.
- Matrix dimension mismatch during backpropagation.
- Updating weights in the wrong direction.
- Incorrect learning rate value.
- Errors in bias updates.

Weight update rule:

$$w_{new} = w_{old} - \eta \frac{\partial L}{\partial w}$$

If the gradient sign is incorrect, the model moves away from the minimum loss instead of toward it.

b) Explain the role of activation functions in gradient flow.

Answer :

Activation functions introduce non-linearity into neural networks and influence how gradients flow during backpropagation.

Common activation functions:

Sigmoid Function

$$f(x) = \frac{1}{1 + e^{-x}}$$

ReLU Function

$$f(x) = \max(0, x)$$

Roles of activation functions:

- Enable learning of complex patterns.
- Control the flow of gradients through layers.
- Affect training speed and convergence.
- Determine whether gradients remain large enough for learning.

Without activation functions, a neural network behaves like a simple linear model regardless of the number of layers.

c) Analyze how vanishing or exploding gradients affect training.

Answer :

Vanishing Gradient Problem

Occurs when gradients become extremely small during backpropagation.

Effects:

- Early layers receive very small updates.
- Learning becomes very slow.
- Network fails to learn deep representations.
- Training may stop improving.

Example:

$1 \rightarrow 0.5 \rightarrow 0.1 \rightarrow 0.01 \rightarrow 0.0001$

Exploding Gradient Problem

Occurs when gradients become extremely large.

Effects:

- Weight updates become unstable.
- Loss function increases rapidly.
- Training diverges.
- Numerical overflow may occur

d) Suggest techniques to fix the issue (e.g., normalization, initialization).

Answer :

Several techniques can improve gradient flow and stabilize training:

1. Proper Weight Initialization

Xavier Initialization

- Suitable for sigmoid and tanh activations.

He Initialization

- Suitable for ReLU activation.

Benefits:

- Maintains stable gradient values.
- Improves convergence.

2. Batch Normalization

- Normalizes layer inputs.
- Reduces internal covariate shift.
- Accelerates training.

3. Gradient Clipping

- Limits excessively large gradients.
- Prevents exploding gradient problems.

4. Use ReLU Activation

- Reduces vanishing gradient issues.
- Speeds up learning.

5. Adjust Learning Rate

- Lower learning rates improve stability

4. Gradient Descent vs SGD Error Investigation

A model trained using Gradient Descent converges very slowly, while the same model with Stochastic Gradient Descent shows unstable loss fluctuations.

Tasks:

a) Explain why batch gradient descent is slow in large datasets.

Answer :

Batch Gradient Descent computes the gradient using the entire training dataset before updating the model parameters.

$$w_{new} = w_{old} - \eta \nabla J(w)$$

where:

- w = model parameters
- η = learning rate
- $\nabla J(w)$ = gradient of the loss function

Reasons for slow convergence:

- Every update requires processing all training samples.
- High computational cost for large datasets.
- Increased memory usage.
- More time required per iteration.
- Training becomes inefficient when millions of samples are present.

As dataset size increases, each update becomes slower, increasing overall training time.

b) Analyze the cause of fluctuations in SGD.

Answer :

Stochastic Gradient Descent (SGD) updates model parameters using only one training sample at a time.

$$w_{new} = w_{old} - \eta \nabla J_i(w)$$

where:

$$\nabla J_i(w)$$

is the gradient computed from a single sample.

Causes of fluctuations:

- Each sample produces a different gradient.
- Gradient estimates are noisy.
- Weight updates vary significantly.
- Loss function may increase or decrease unpredictably.
- The optimization path becomes unstable.

c) Compare the error behavior of Mini-Batch Gradient Descent.

Answer :

Mini-Batch Gradient Descent uses a small subset of training samples (batch) for each update.

Example:

Batch Size = 32 or 64 samples

Error behavior:

- More stable than SGD.
- Faster than Batch Gradient Descent.
- Reduced noise in gradient estimates.
- Smoother convergence.
- Better computational efficiency.

Comparison:

Method	Error Behavior
Batch Gradient Descent	Smooth and stable
SGD	Noisy and fluctuating
Mini-Batch Gradient Descent	Moderately smooth and stable
Mini-Batch Gradient Descent balances speed and stability.	

d) Recommend the best approach and justify with error trade-offs.

Answer :

The recommended approach is Mini-Batch Gradient Descent.

Reasons:

- Faster than Batch Gradient Descent.
- More stable than SGD.
- Efficient memory usage.
- Suitable for large datasets.
- Works efficiently with GPUs.
- Provides good convergence speed and accuracy.

Trade-offs:

Method	Advantage	Limitation
Batch GD	Stable convergence	Slow training
SGD	Fast updates	High fluctuations
Mini-Batch GD	Balance of speed and stability	Requires batch-size tuning

Therefore, Mini-Batch Gradient Descent offers the best compromise between computational efficiency, convergence speed, and prediction accuracy.

5. Curse of Dimensionality Error Analysis

A machine learning model built using high-dimensional data performs well on training data but poorly on test data due to overfitting, related to the Curse of Dimensionality.

Tasks:

a) Explain how high dimensionality increases model error.

Answer :

High dimensionality means the dataset contains a very large number of features. As the number of dimensions increases:

- The feature space becomes sparse.
- More training data is required to learn patterns effectively.
- Model complexity increases.
- Computational cost becomes higher.
- The risk of overfitting increases.

As a result, the model may memorize training data instead of learning general patterns, leading to higher prediction errors on unseen data.

b) Identify signs of overfitting in this scenario.

Answer :

Overfitting occurs when a model performs extremely well on training data but poorly on test data.

Signs of overfitting include:

- Very high training accuracy.
- Low testing accuracy.
- Large gap between training and testing performance.
- Poor generalization to new data.
- Model learns noise and irrelevant patterns.

Example:

Training Accuracy = 99%

Testing Accuracy = 70%

The large difference between training and testing accuracy indicates overfitting.

c) Analyze why distance-based learning fails in high dimensions.

Answer :

Distance-based algorithms such as K-Nearest Neighbors (KNN) and K-Means Clustering rely on distance measures to identify similar data points.

In high-dimensional spaces:

- Distances between points become very similar.
- The difference between nearest and farthest neighbors decreases.
- Similarity measurements become unreliable.
- Data points become sparse.

Mathematically:

$$Distance_{Nearest} \approx Distance_{Farthest}$$

Effects:

- Nearest neighbors become difficult to identify.
- Clustering quality decreases.
- Classification accuracy drops.
- Model performance becomes unreliable.

Therefore, distance-based learning methods become less effective as dimensionality increases.

d) Suggest dimensionality reduction or regularization techniques to improve performance.

Answer :

Dimensionality Reduction Techniques

1. Principal Component Analysis (PCA)

- Converts original features into a smaller set of principal components.
- Retains most important information.
- Reduces noise and redundancy.

2. Linear Discriminant Analysis (LDA)

- Reduces dimensions while preserving class separability.
- Improves classification performance.

3. Autoencoders

- Neural network-based dimensionality reduction technique.
- Learns compact feature representations.

Regularization Techniques

1. L1 Regularization (LASSO)

$$Loss + \lambda \sum |w|$$

Benefits:

- Removes less important features.
- Produces simpler models.

2. L2 Regularization (Ridge)

$$Loss + \lambda \sum w^2$$

Benefits:

- Prevents large weight values.
- Reduces model complexity.

3. Dropout

- Randomly deactivates neurons during training.
- Reduces overfitting.
- Improves generalization.

These techniques reduce complexity and improve model performance on unseen data.

Conclusion

The Curse of Dimensionality is a significant challenge in machine learning because high-dimensional datasets increase model complexity, make data sparse, and raise the risk of overfitting. Overfitted models achieve excellent performance on training data but fail to generalize to test data. Distance-based algorithms become less effective because distances lose their discriminative power in high-dimensional spaces. Techniques such as Principal Component Analysis (PCA), Linear Discriminant Analysis (LDA), Autoencoders, L1/L2 Regularization, and Dropout help reduce dimensionality and control model complexity. Therefore, dimensionality reduction and regularization are essential for improving generalization, reducing overfitting, and building accurate machine learning models.

